C:\hpc-23-feb-2026\.\16\eg141.cpp

```
// winpthread api has a bug in case of pthread_exit
// does not unwind and destruct local object/variables
// we are not worried about winpthread bug, as we will be moving
// on everything to linux which unwinds and local objects in the chain of
// functions involved are destroyed
#include<iostream>
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
using namespace std;
class aaa
{
public:
aaa()
{
cout<<"aaa Constructor"<<endl;
}
~aaa()
{
cout<<"aaa Destructor"<<endl;
}
};
class bbb
{
public:
bbb()
{
cout<<"bbb Constructor"<<endl;
}
~bbb()
{
cout<<"bbb Destructor"<<endl;
}
};
int divide(int numerator,int denominator)
{
aaa a;
if(denominator==0) pthread_exit(NULL);
return numerator/denominator;
}
void * do_something(void *ptr)
{
bbb b;
int rst;
int *p;
int *m;
p=(int *)ptr;
rst=divide(p[0],p[1]);
```

```
m=(int *)malloc(sizeof(int));
*m=rst;
pthread_exit((void *)m);
return NULL;
}
int main()
{
int e,f;
int x[2];
int *g;
pthread_t pid;
int i;
printf("Enter a number : ");
scanf("%d",&e);
printf("Enter another number : ");
scanf("%d",&f);
x[0]=e;
x[1]=f;
i=pthread_create(&pid,NULL,do_something,(void *)x);
if(i)
{
printf("Unable to created threads\n");
return 0;
}
pthread_join(pid,(void **)&g);
if(g==NULL)
{
printf("Unable to divide\n");
}
else
{
printf("Result : %d\n",*g);
free(g);
}
return 0;
}
```

C:\hpc-23-feb-2026\.\16\eg142.cpp

```
#include<pthread.h>
#include<iostream>
#include<unistd.h> // for sleep function
using namespace std;
struct cmn
{
char str[11];
};
struct arguments
{
struct cmn *c;
char str[11];
};
void * do_something(void *args)
{
struct arguments *a;
a=(struct arguments *)args;
strcpy(a->c->str,a->str);
cout<<a->c->str<<endl;
sleep(1);
cout<<a->c->str<<endl;
pthread_exit(NULL);
return NULL; // unreachable
}
int main()
{
struct cmn c;
pthread_t tid1,tid2,tid3;
int i1,i2,i3;
struct arguments arg1,arg2,arg3;
arg1.c=&c;
strcpy(arg1.str,"Hello");
arg2.c=&c;
strcpy(arg2.str,"Boys");
arg3.c=&c;
strcpy(arg3.str,"Girls");
i1=pthread_create(&tid1,NULL,do_something,(void *)&arg1);
i2=pthread_create(&tid2,NULL,do_something,(void *)&arg2);
i3=pthread_create(&tid3,NULL,do_something,(void *)&arg3);
if(i1==0) pthread_join(tid1,NULL);
if(i2==0) pthread_join(tid2,NULL);
if(i3==0) pthread_join(tid3,NULL);
return 0;
}
```

C:\hpc-23-feb-2026\.\16\eg143.cpp

```
#include<pthread.h>
#include<iostream>
#include<unistd.h> // for sleep function
using namespace std;
struct cmn
{
char str[11];
};
struct arguments
{
struct cmn *c;
char str[11];
pthread_mutex_t *mtx;
};
void * do_something(void *args)
{
struct arguments *a;
a=(struct arguments *)args;
pthread_mutex_lock(a->mtx);
strcpy(a->c->str,a->str);
cout<<a->c->str<<endl;
sleep(1);
cout<<a->c->str<<endl;
pthread_mutex_unlock(a->mtx);
pthread_exit(NULL);
return NULL; // unreachable
}
int main()
{
struct cmn c;
pthread_t tid1,tid2,tid3;
int i1,i2,i3;
pthread_mutex_t mtx;
struct arguments arg1,arg2,arg3;
// it is essential to initialize mutex
pthread_mutex_init(&mtx,NULL);
arg1.c=&c;
strcpy(arg1.str,"Hello");
arg1.mtx=&mtx;
arg2.c=&c;
strcpy(arg2.str,"Boys");
arg2.mtx=&mtx;
arg3.c=&c;
arg3.mtx=&mtx;
strcpy(arg3.str,"Girls");
i1=pthread_create(&tid1,NULL,do_something,(void *)&arg1);
i2=pthread_create(&tid2,NULL,do_something,(void *)&arg2);
i3=pthread_create(&tid3,NULL,do_something,(void *)&arg3);
```

```
if(i1==0) pthread_join(tid1,NULL);
if(i2==0) pthread_join(tid2,NULL);
if(i3==0) pthread_join(tid3,NULL);
pthread_mutex_destroy(&mtx);
return 0;
}
```

Thinking Machines Classroom Codes